

G06 Gate A - NorthStar Logistics Shipments

Gate A available after M17 completion.

Independent protected assessment. Keep Teaching Books, Reviews, tutorial videos and AI solving closed during the attempt. Official product help may be used only to verify syntax/current platform behavior, not to obtain a solution.

Files

01_GATE_A/G06_GATE_A_SOURCE/source_records.csv + requirements.json + incident.txt + evidence_template.json

Pass boundary

K1-K7 must each be demonstrated with concrete evidence; no unresolved critical data-quality, PII/secret, access, deployment, rollback, freshness or recovery failure may remain. No percentage-based shortcut.

K1 - Fabric/cloud workload design

Translate the requirements into an architecture. Choose ingestion, storage/serving, compute/orchestration and identity boundaries. Explain why each choice fits the workload rather than naming Fabric features from memory.

K2 - Ingestion and cloud operations

Define batch/incremental behavior, checkpoint/state ownership, monitoring, performance and final reconciliation. State what evidence would prove the design in a real Fabric workspace and what remains simulated/not run.

K3 - DataOps / CI-CD

Define tests, blocking quality gates, CI trigger, dev-test-prod promotion and rollback. Include one deliberately failing data/change condition and explain why deployment must stop.

K4 - Security / governance

Protect the PII field, define secret handling and least-privilege RBAC, then explain lineage/governance evidence. A raw PII serving output or hard-coded secret is a critical failure.

K5 - Production reliability

Define freshness/SLA signals, alert owner, incident steps, recovery, validation and communication. Include cost guardrails and explain safe IaC/state/change management at foundation level.

K6 - Architecture trade-offs

Defend batch vs streaming, warehouse vs lakehouse, managed vs build, scale/bottleneck and failure-domain choices. Reject at least two plausible alternatives with reasons.

K7 - Integrated defense

Complete the evidence template and give a concise architecture defense. Separate what you actually executed from what you only designed. Explain one promotion-foundation concept (CDC/contracts/Vault/Mesh/Kubernetes) only where it genuinely helps this scenario.

Required final evidence

Save the completed evidence_template.json (or an equivalent structured packet), your source-profile counts/totals, quality-gate result, architecture diagram/description, security decisions, incident/recovery plan, final reconciliation, and a runtime-truth statement. Preserve the first attempt even if it fails.